codetopia | Community

Codetopia Mentorship Program - 2026

Git & GitHub

Session 3 : Reading & Navigating History

What we will cover today

- Understanding HEAD and what it points to
- Navigating to past commits safely
- Using git show and git diff to inspect changes
- Detached HEAD state — what it is and why it's safe
- Session Project: History Detective

Quick Review From Previous Session

- What are the 3 Git states, and what command moves a file from the Working Directory into the Staging Area?
- You run git status and see a file name in red. What does red mean, and what should you do next?
- What is the difference between a feat: commit and a fix: commit? Give one example of each.
- You made a commit with the message ”stuff”. How do you fix the message without creating a new commit?

odetopi | Community

The Core Workflow

Understanding HEAD

What is HEAD?

- HEAD is a pointer — it marks where you currently are in your project's history
- Normally HEAD points to the tip of your current branch
- Moving HEAD backward lets you explore the past without changing anything

Understanding HEAD

Term	What it points to
HEAD	Your current position (usually the latest commit on active branch)
main/master	The latest commit on the main branch
a3f9c2d	A specific snapshot in history

Navigating History Safely

Useful log commands

```
git log --oneline --graph --all
git show <commit-hash>          # view one commit in detail
git diff <hash1> <hash2>        # compare two commits
```

Checking out a past commit (read-only time travel)

```
git checkout <commit-hash>      # enter detached HEAD state
git checkout main               # return to the present
```

Detached HEAD : what it means

You are viewing a past commit. You are NOT on any branch. You CAN explore and even make commits here — but they will be lost unless you create a new branch first. To return safely: git checkout main

odetopi

Session Project

Commit History Journal

Session Project

Your deliverables

1. Run `git log --oneline --graph --all` on your Week 1 project screenshot it
2. Use `git show <commit-hash>` on 2 commits; write one sentence explaining each
3. Run `git diff <hash1> <hash2>` to compare two commits — screenshot the output
4. Use `git checkout <hash>` to travel to a past commit, explore, then return to main
5. Write 3 sentences explaining: what is `HEAD` and what is detached `HEAD` state?

Session Project - Instructions

Your deliverables

1. Run `git log --oneline --graph --all` on your Week 1 project screenshot it
2. Use `git show <commit-hash>` on 2 commits; write one sentence explaining each
3. Run `git diff <hash1> <hash2>` to compare two commits — screenshot the output
4. Use `git checkout <hash>` to travel to a past commit, explore, then return to main
5. Write 3 sentences explaining: what is `HEAD` and what is detached `HEAD` state?

References & Further Reading

Resources

Text Resource

- Atlassian: Git Log Guide
  Advanced Git Log | Atlassian Git Tutorial

Video Resource

Fireship: Git It (first 5 min on history)
https://www.youtube.com/watch?v=HkdAHXoRtos